Data Compression Course Project

Implementation of BZip2 Compression Algorithm

Course Instructor: Dr. Faisal Aslam
Date: April 14, 2026

Contents

1 Introduction **3**
1.1 Project Overview . . . 3
1.2 Project Timeline . . . 3
1.3 Evaluation Criteria . . . 3

2 System Architecture **4**
2.1 Overall Block Diagram . . . 4
2.2 Configuration Management . . . 4

3 Stage 1 Implementation (1.5 Weeks) **5**
3.1 Block Division and File Handling . . . 5
3.1.1 Requirements . . . 5
3.1.2 Data Structures . . . 5
3.1.3 Function Prototypes . . . 5
3.2 RLE-1 Implementation . . . 6
3.2.1 Description . . . 6
3.2.2 Function Prototypes . . . 6
3.3 Burrows-Wheeler Transform (BWT) . . . 6
3.3.1 Description . . . 6
3.3.2 Data Structures . . . 7
3.3.3 Function Prototypes . . . 7
3.4 Stage 1 Evaluation . . . 7

4 Stage 2 Implementation (6 Days) **8**
4.1 Move-to-Front (MTF) Transform . . . 8
4.1.1 Description . . . 8
4.1.2 Function Prototypes . . . 8
4.2 RLE-2 Implementation . . . 8
4.2.1 Description . . . 8
4.2.2 Function Prototypes . . . 8
4.3 Stage 2 Evaluation . . . 9

5 Stage 3 Implementation (1 Week) **9**
5.1 Canonical Huffman Coding . . . 9
5.1.1 Description . . . 9
5.1.2 Data Structures . . . 9
5.1.3 Function Prototypes . . . 9

6 Build System and Platform Support **11**
6.1 Makefile Requirements . . . 11

7 Performance Evaluation and Benchmarking **11**
7.1 Benchmark Datasets . . . 11
7.2 Performance Metrics . . . 11
7.3 Automated Results Generation . . . 12

8 Extra Features and Enhancements (10% Marks) 12
8.1 Enhanced RLE Versions (Extra Marks) 12
8.2 Suffix Array-based BWT (Extra Marks) 12
8.3 Alternative Entropy Coding 12

9 Submission Guidelines 13
9.1 GitHub Repository Structure 13
9.2 README.md Requirements 13
9.3 Graph Generation 13

10 Important Notes for Students 14

11 Conclusion 14

1 Introduction

1.1 Project Overview

This project involves implementing a simplified version of the BZip2 compression algorithm. The project spans three weeks and is completed by a team of three individuals. The implementation includes all major components of BZip2: Run-Length Encoding (RLE), Burrows-Wheeler Transform (BWT), Move-to-Front (MTF) transform, and Huffman coding.

Important: Students must write their own code. Only function prototypes and struct definitions are provided as guidance. No implementation code is given.

1.2 Project Timeline

- **Total Duration:** 3 Weeks
- **Stage 1:** 1.5 Weeks (50% of project grade)
- **Stage 2:** 6 Days (25% of project grade)
- **Stage 3:** 1 Week (25% of project grade)

1.3 Evaluation Criteria

- Each stage includes evaluation of both encoding and decoding
- Extra features: 10% additional marks
- Performance beating latest BZip2: 10% additional marks (max 2 groups)

2 System Architecture

2.1 Overall Block Diagram

```
Encoding Pipeline:
================
Input File
    |
    v
Block Division
    |
    v
RLE-1 Encoding
    |
    v
BWT
    |
    v
MTF
    |
    v
RLE-2 Encoding
    |
    v
Huffman Encoding
    |
    v
Compressed Output
```

Figure 1: Encoding Pipeline Architecture

2.2 Configuration Management

The system uses a configuration file named `config.ini` with the following structure:

```
[General]
block_size = 500000  # Bytes (100KB to 900KB)
rle1_enabled = true
bwt_type = matrix   # matrix or suffix_array
mtf_enabled = true
rle2_enabled = true
huffman_enabled = true

[Performance]
benchmark_mode = false
output_metrics = true

[Paths]
input_directory = ./benchmarks/
```

```
output_directory = ./results/
```

Listing 1: Configuration File Format

3 Stage 1 Implementation (1.5 Weeks)

3.1 Block Division and File Handling

3.1.1 Requirements

- Support for very large files
- Configurable block size (100KB - 900KB)
- Configuration file integration

3.1.2 Data Structures

```c
/*
 * Structure to hold a single block of data
 */
typedef struct {
    unsigned char *data;      // Pointer to block data
    size_t size;              // Current size of block
    size_t original_size;     // Original size before compression
} Block;

/*
 * Structure to manage multiple blocks
 */
typedef struct {
    Block *blocks;            // Array of blocks
    int num_blocks;           // Number of blocks
    size_t block_size;        // Configurable block size
} BlockManager;
```

Listing 2: Block Division Structures (Provided)

3.1.3 Function Prototypes

```c
/*
 * Reads input file and divides into blocks
 * @param filename: Input file path
 * @param block_size: Size of each block in bytes
 * @return: BlockManager structure containing all blocks
 */
BlockManager* divide_into_blocks(const char *filename, size_t
    block_size);

/*
 * Reassembles blocks back into original file
 * @param manager: BlockManager containing processed blocks
 * @param output_filename: Path for output file
 * @return: 0 on success, -1 on failure
```

```c
 */
int reassemble_blocks(BlockManager *manager, const char *
    output_filename);

/*
 * Frees memory allocated for BlockManager
 * @param manager: Pointer to BlockManager to free
 */
void free_block_manager(BlockManager *manager);
```

Listing 3: Block Management Functions

3.2 RLE-1 Implementation

3.2.1 Description

Run-Length Encoding replaces sequences of repeated characters with a count and the character.

Example:

```
Input:  ABBBCCCCD
Output: A3B4C1D
```

3.2.2 Function Prototypes

```c
/*
 * Encodes data using Run-Length Encoding
 * @param input: Input byte array
 * @param len: Length of input array
 * @param output: Output buffer (must be pre-allocated)
 * @param out_len: Pointer to store output length
 */
void rle1_encode(unsigned char *input, size_t len,
                 unsigned char *output, size_t *out_len);

/*
 * Decodes RLE-1 encoded data
 * @param input: Encoded byte array
 * @param len: Length of encoded data
 * @param output: Output buffer for decoded data
 * @param out_len: Pointer to store decoded length
 */
void rle1_decode(unsigned char *input, size_t len,
                 unsigned char *output, size_t *out_len);
```

Listing 4: RLE-1 Functions

3.3 Burrows-Wheeler Transform (BWT)

3.3.1 Description

The matrix-based BWT creates all cyclic rotations of the input string, sorts them lexicographically, and outputs the last column.

3.3.2 Data Structures

```c
/*
 * Structure for rotation in BWT
 */
typedef struct {
    char *rotation;     // Rotation string
    int index;          // Original index
} Rotation;
```

Listing 5: BWT Structures

3.3.3 Function Prototypes

```c
/*
 * Compares two rotations for sorting
 * @param a: First rotation
 * @param b: Second rotation
 * @return: Comparison result (-1, 0, 1)
 */
int compare_rotations(const void *a, const void *b);

/*
 * Forward BWT transform
 * @param input: Input byte array
 * @param len: Length of input
 * @param output: Output buffer for BWT result
 * @param primary_index: Pointer to store primary index
 */
void bwt_encode(unsigned char *input, size_t len,
                unsigned char *output, int *primary_index);

/*
 * Inverse BWT transform
 * @param input: BWT encoded data
 * @param len: Length of encoded data
 * @param primary_index: Primary index from encoding
 * @param output: Output buffer for original data
 */
void bwt_decode(unsigned char *input, size_t len,
                int primary_index, unsigned char *output);
```

Listing 6: BWT Functions

3.4 Stage 1 Evaluation

Both encoding and decoding for Stage 1 components (RLE-1 and BWT) must be fully functional. The evaluation tests:

- Correctness of RLE-1 encoding/decoding
- Correctness of BWT forward/inverse transform
- Block division and reassembly
- Configuration file parsing

4 Stage 2 Implementation (6 Days)

4.1 Move-to-Front (MTF) Transform

4.1.1 Description

MTF encodes data by maintaining a list of symbols and outputting the index of each symbol, then moving it to the front.

4.1.2 Function Prototypes

```c
/*
 * Forward MTF transform
 * @param input: Input byte array
 * @param len: Length of input
 * @param output: Output buffer for MTF indices
 */
void mtf_encode(unsigned char *input, size_t len,
                unsigned char *output);

/*
 * Inverse MTF transform
 * @param input: MTF encoded indices
 * @param len: Length of input
 * @param output: Output buffer for decoded data
 */
void mtf_decode(unsigned char *input, size_t len,
                unsigned char *output);
```

Listing 7: MTF Functions

4.2 RLE-2 Implementation

4.2.1 Description

RLE-2 specifically targets the output of MTF, which contains many zeros and small numbers. This is a specialized RLE for the MTF output.

4.2.2 Function Prototypes

```c
/*
 * Encodes MTF output using specialized RLE
 * @param input: MTF output array
 * @param len: Length of input
 * @param output: Output buffer
 * @param out_len: Pointer to store output length
 */
void rle2_encode(unsigned char *input, size_t len,
                 unsigned char *output, size_t *out_len);

/*
 * Decodes RLE-2 encoded data
 * @param input: RLE-2 encoded data
 * @param len: Length of encoded data
```

```
 * @param output: Output buffer for MTF data
 * @param out_len: Pointer to store output length
 */
void rle2_decode(unsigned char *input, size_t len,
                 unsigned char *output, size_t *out_len);
```

Listing 8: RLE-2 Functions

4.3 Stage 2 Evaluation

The complete pipeline (RLE-1 → BWT → MTF → RLE-2) must work for both encoding and decoding.

5 Stage 3 Implementation (1 Week)

5.1 Canonical Huffman Coding

5.1.1 Description

Canonical Huffman coding ensures efficient tree representation by storing only code lengths, not the full tree.

5.1.2 Data Structures

```
/*
 * Huffman code representation
 */
typedef struct {
    unsigned short code;    // Huffman code
    unsigned char length;   // Code length in bits
} HuffmanCode;

/*
 * Huffman tree node
 */
typedef struct Node {
    unsigned char symbol;   // Byte value (0-255)
    int freq;               // Frequency count
    struct Node *left;      // Left child
    struct Node *right;     // Right child
} HuffmanNode;
```

Listing 9: Huffman Data Structures

5.1.3 Function Prototypes

```
/*
 * Builds Huffman tree from frequency counts
 * @param frequencies: Array of 256 frequency counts
 * @param root: Pointer to store root of Huffman tree
 */
void build_huffman_tree(int *frequencies, HuffmanNode **root);

```

```c
/*
 * Generates canonical Huffman codes from tree
 * @param root: Root of Huffman tree
 * @param codes: Array of 256 to store generated codes
 */
void generate_canonical_codes(HuffmanNode *root,
                              HuffmanCode *codes);

/*
 * Encodes data using Huffman coding
 * @param input: Input byte array
 * @param len: Length of input
 * @param output: Output buffer for compressed data
 * @param out_len: Pointer to store output length
 */
void huffman_encode(unsigned char *input, size_t len,
                    unsigned char *output, size_t *out_len);

/*
 * Decodes Huffman encoded data
 * @param input: Huffman encoded data
 * @param len: Length of encoded data
 * @param output: Output buffer for decoded data
 * @param out_len: Pointer to store output length
 */
void huffman_decode(unsigned char *input, size_t len,
                    unsigned char *output, size_t *out_len);

/*
 * Writes Huffman header (code lengths) to output
 * @param codes: Array of Huffman codes
 * @param output: Output buffer
 * @param out_len: Pointer to update with header size
 */
void write_header(HuffmanCode *codes, unsigned char *output,
                  size_t *out_len);

/*
 * Encodes data using generated codes
 * @param input: Input data
 * @param len: Length of input
 * @param codes: Huffman codes for each symbol
 * @param output: Output buffer
 * @param out_len: Pointer to update with encoded size
 */
void encode_data(unsigned char *input, size_t len,
                 HuffmanCode *codes, unsigned char *output,
                 size_t *out_len);
```

Listing 10: Huffman Functions

6 Build System and Platform Support

6.1 Makefile Requirements

The project must include a cross-platform Makefile supporting both Linux and Windows. The Makefile should have the following targets:

```
# Required Makefile targets:
# all          - Compile the complete project
# clean        - Remove object files and executable
# windows      - Cross-compile for Windows platform

# Example structure (students must complete):
CC = gcc
CFLAGS = -Wall -O2
TARGET = bzip2_impl
SOURCES = main.c rle.c bwt.c mtf.c huffman.c config.c

# Platform detection and flags must be implemented
# Windows cross-compilation must be supported
```

Listing 11: Makefile Structure (Students must implement)

7 Performance Evaluation and Benchmarking

7.1 Benchmark Datasets

Students must use the following publicly available benchmark files for testing:

Table 1: Benchmark Datasets

Dataset	**Size**	**Type**
Canterbury Corpus	Various	Mixed
Calgary Corpus	Various	Mixed
Silesia Corpus	Various	Mixed
Large Text Files	10MB-100MB	Text
Binary Files	10MB-100MB	Binary

7.2 Performance Metrics

Performance is evaluated using a weighted formula:

$$\text{Score} = w_1 \cdot \frac{C_{\text{ref}}}{C} + w_2 \cdot \frac{S}{S_{\text{ref}}}$$

Where:

- C = Compression ratio achieved
- C_{ref} = Reference compression ratio (bzip2)
- S = Speed (MB/s)

- S_{ref} = Reference speed (bzip2)
- w_1, w_2 = Weights (to be determined by instructor)

7.3 Automated Results Generation

Students must implement a script to automatically generate results in `results.csv` with the following format:

```
# results.csv must contain:
File,Size,BlockSize,CompressionRatio,Time,Memory

# Students must implement their own benchmarking script
```

Listing 12: Required Results Format

8 Extra Features and Enhancements (10% Marks)

These will be 10% marks of the overall project grade.

8.1 Enhanced RLE Versions (Extra Marks)

Students can implement:

- **RLE with run-length thresholds:** Only encode runs longer than a configurable threshold
- **Adaptive RLE:** Dynamically adjust encoding based on data characteristics
- **RLE with entropy coding:** Combine with simple entropy coding for better compression

8.2 Suffix Array-based BWT (Extra Marks)

Faster implementation using suffix array algorithm:

```c
/*
 * Builds suffix array for efficient BWT
 * @param text: Input text
 * @param n: Length of text
 * @return: Suffix array
 */
int *build_suffix_array(unsigned char *text, int n);
```

Listing 13: Suffix Array BWT Prototype

8.3 Alternative Entropy Coding

- **ANS (Asymmetric Numeral Systems):** Modern entropy coder with near-optimal compression
- **Range Coding:** Arithmetic coding alternative with integer precision

9 Submission Guidelines

9.1 GitHub Repository Structure

```
project-bzip2/
|-- src/
|   |-- main.c
|   |-- rle.c
|   |-- bwt.c
|   |-- mtf.c
|   |-- huffman.c
|   `-- config.c
|-- include/
|   `-- *.h
|-- benchmarks/
|   `-- (test files)
|-- results/
|   `-- (output files)
|-- Makefile
|-- config.ini
|-- README.md
`-- report.pdf
```

9.2 README.md Requirements

The README must include:

- Complete feature description
- Implementation details for each stage (written by students)
- Build instructions for Linux and Windows
- Usage examples
- Performance results and graphs
- Team member names and contributions

9.3 Graph Generation

Students must implement a script to generate performance graphs. Example structure:

```python
#!/usr/bin/env python3
# Students must implement their own graph generation
# Should read results.csv and produce graphs

import matplotlib.pyplot as plt
import pandas as pd

# Students implement their own plotting logic
```

Listing 14: Plot Generation Script Structure

10 Important Notes for Students

- **No code implementation is provided** - you must write all functions yourself
- Only struct definitions and function prototypes are given as guidance
- You may discuss algorithms but must write your own code
- Plagiarism will result in grade penalty
- Each team member must contribute to the implementation
- Submit GitHub repository link with all required files

11 Conclusion

This project requires implementing a complete BZip2-like compression pipeline with configurable parameters, cross-platform support, and comprehensive benchmarking capabilities. Students must write all code themselves based on the provided specifications and prototypes.

References

[1] Seward, J. (1996). The bzip2 and libbzip2 official home page.

[2] Burrows, M., & Wheeler, D. J. (1994). A block-sorting lossless data compression algorithm.

[3] Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes.

[4] Manber, U., & Myers, G. (1993). Suffix arrays: a new method for on-line string searches.